

一、审计经验较少或主要是代码审查，可以这样回答：

"我参与过合约的代码审查和安全检查工作。虽然不是完整的第三方审计，但我们在开发流程中有严格的安全检查环节。

我主要负责在代码提交前进行安全审查，会关注几个重点：检查是否使用了SafeMath或Solidity 0.8.x的自动溢出检查、验证访问控制修饰符是否正确使用、审查外部调用是否遵循了Checks-Effects-Interactions模式、检查是否有潜在的重入风险。

我们团队会使用一些自动化工具辅助，比如Slither做静态分析，然后结合OpenZeppelin的合约作为安全基准。对于关键的资金相关函数，我们会进行更细致的人工检查，并编写详细的单元测试覆盖各种边界情况。

另外，我也研究过一些知名的安全事件案例，比如DAO攻击、Poly Network被盗等，从中学习攻击手法和防御策略。我认为合约审计不仅是技术问题，更重要的是建立安全思维，要站在攻击者的角度去思考可能的漏洞点。"

二、你自己有审计的经验，可以这样回答：

"是的，我参与过智能合约的审计工作。在我们团队的实践中，审计流程主要分为几个阶段：

前期准备阶段，我们会先收集完整的合约代码、技术文档和业务逻辑说明，了解项目的代币经济模型、权限设计和核心业务流程。

静态分析阶段，我们会使用工具如Slither、Mythril等进行自动化扫描，主要检查重入攻击、整数溢出、权限控制缺陷等常见漏洞。

手工审计阶段是最核心的部分，我会重点关注几个方面：一是业务逻辑漏洞，比如价格操纵、闪电贷攻击向量；二是权限管理，检查owner权限是否过大、多签机制是否合理；三是经济模型安全性，特别是涉及资金流转的函数；四是外部调用安全，检查与其他合约或预言机交互时的风险点。

测试验证阶段，我们会编写针对性的测试用例，特别是边界条件和异常场景的测试，同时会在测试网进行实际部署验证。

出具报告阶段，我们会将发现的问题按严重程度分级：Critical（严重）、High（高危）、Medium（中危）、Low（低危）和Info（信息级），并给出具体的修复建议。

在审计过程中，我发现最常见的问题包括：权限控制不当、重入攻击风险、价格预言机操纵、未检查返回值、时间戳依赖等。我们通常会要求项目方修复所有Critical和High级别的问题后，进行二次审计确认。"

三、与第三方审计公司（如CertiK、SlowMist、PeckShield等）合作的完整流程的完整流程

第一阶段：审计前准备（1-2周）

1. 确定审计范围

- 明确需要审计的合约清单（主合约、代理合约、依赖库等）

- 确定代码冻结时间点（通常是某个Git commit hash）
- 说明哪些是核心合约，哪些是辅助合约
- 明确是否包含前端交互安全审查

2. 准备审计材料包

你需要提供给审计公司：

- 完整的源代码（GitHub私有仓库授权访问权限）
- 技术白皮书或项目文档
- 合约架构图和业务流程图
- 详细的功能说明文档，包括每个主要函数的作用
- 已知的风险点说明
- 测试用例和测试覆盖率报告
- 依赖的外部合约地址和接口说明（如预言机、DEX等）
- 代币经济模型文档
- 部署脚本和配置参数说明

3. 商务对接

- 获取报价（通常按代码行数或合约复杂度计费）
- 确定审计周期（一般2-4周）
- 签订保密协议（NDA）
- 明确交付物：审计报告、修复验证报告、是否提供徽章等

第二阶段：审计进行中（2-4周）

1. 启动会议

- 向审计团队讲解项目背景和业务逻辑
- 说明重点关注的风险区域
- 介绍代码架构和设计思路
- 回答审计师的初步疑问
- 确定沟通渠道（通常是Telegram群或企业微信）

2. 日常配合

- 指定专人对接审计团队（通常是技术负责人或架构师）
- 及时回答审计师提出的技术问题
- 解释业务逻辑的设计意图
- 提供额外的文档或说明
- 响应时间尽量控制在24小时内

3. 中期沟通

- 审计进行到一半时，审计公司通常会有一次中期沟通
- 他们会告知已发现的主要问题类型
- 你可以提前了解大致的问题严重程度
- 这时可以开始准备修复方案的讨论

第三阶段：问题修复（1-2周）

1. 收到初步审计报告

- 仔细阅读所有发现的问题
- 按严重级别分类：Critical、High、Medium、Low、Informational
- 团队内部评估每个问题的合理性
- 制定修复优先级

2. 问题讨论

- 对于不理解的问题，与审计师深入沟通
- 如果认为某些问题是误报，提供技术依据说明
- 对于确实存在的问题，讨论最佳修复方案
- 有些问题可能有多种修复方式，选择最适合的

3. 代码修复

- 优先修复Critical和High级别问题
- 所有修复必须有代码审查
- 修复后编写针对性测试用例
- 保持代码变更的可追溯性（清晰的commit message）
- 准备修复说明文档

第四阶段：复审与交付（1周）

1. 提交修复代码

- 提供修复后的代码和新的commit hash
- 提交详细的修复说明文档
- 说明哪些问题已修复、如何修复
- 说明哪些问题暂不修复及原因（如果有）

2. 复审流程

- 审计公司验证修复效果
- 确认问题是否真正解决
- 检查修复是否引入新的问题
- 通常需要3-5个工作日

3. 最终报告

- 收到包含修复验证的最终报告
- 报告会标注每个问题的状态：已修复、部分修复、未修复
- 审计公司可能会给出安全评分
- 获得审计徽章（用于官网展示）

第五阶段：后续事项

1. 报告使用

- 审计报告可以公开，增强用户信任
- 但要注意有些敏感信息可能需要脱敏
- 可以在官网、白皮书、社交媒体上展示审计徽章

2. 持续安全

- 如果后续有重大代码变更，需要进行增量审计
- 建立与审计公司的长期合作关系
- 有些公司提供应急响应服务

关键注意事项

时间规划建议：

- 产品上线前至少预留6-8周时间做审计
- 不要在审计期间进行大规模代码重构
- 预留充足的修复和复审时间

成本控制：

- 代码质量越高，审计越快越便宜
- 提前做好自测和工具扫描，减少低级错误
- 清晰的文档可以减少沟通成本

常见坑：

- 不要在收到报告后才意识到问题严重性，要持续沟通
- 不要盲目修复，有些"问题"可能是业务设计需要
- 不要隐瞒已知风险，审计师最终都会发现
- 确保代码冻结，审计期间不要继续开发新功能

面试时可以讲的点：

"我们之前与CertiK合作过一次审计，整个流程大概6周。我负责技术对接，最大的体会是准备工作很重要。我们提前整理了详细的技术文档和架构图，第一次沟通就很顺畅。

审计过程中他们发现了我们一个重要的价格预言机操纵风险，当时我们用的是单一数据源，他们建议使用Chainlink的多源聚合或者加入TWAP机制。还有一个是权限管理问题，我们的owner权限过大，他们建议引入时间锁和多签。

修复阶段我们用了10天，所有Critical和High问题都解决了，复审很快就通过了。最终报告显示安全评分是85分，对我们后续融资和上线帮助很大。

我认为审计不只是走流程，更重要的是通过专业团队的视角发现盲区，提升整体安全意识。"